

Base de Datos I

Trabajo Práctico Especial

1er Cuatrimestre 2026

1. Objetivo

El objetivo de este Trabajo Práctico Especial es aplicar los conceptos de SQL Avanzado (PSM, Triggers) vistos a lo largo del curso, para implementar funcionalidades y restricciones no disponibles de forma estándar (que no pueden resolverse con Primary Keys, Foreign Keys, etc.).

2. Modalidad

El Trabajo Práctico estará disponible en el Campus a partir del jueves 11/06/2026.

Se incluyen junto con el enunciado 2 archivos: **subasta.csv** y **oferta.csv**.

El TP deberá realizarse en grupos de 4 alumnos (a excepción de 1 grupo de 3 alumnos) y entregarse a través de la plataforma Campus ITBA hasta el jueves 18/06/2026 a las 23:59.

3. Descripción del Trabajo

Se tiene un esquema de base de datos para un sistema de subastas online. Los usuarios pueden actuar como vendedores (publicando subastas) o como oferentes (pujando por ítems en subastas abiertas). Cada subasta tiene un precio base, un incremento mínimo entre ofertas, y un período de actividad acotado entre una fecha de inicio y una fecha de cierre.

Las reglas de negocio del dominio son las siguientes:

- Una subasta tiene un único vendedor (el usuario que la publica)
- Solo se aceptan ofertas mientras la subasta está activa, es decir, entre fecha_inicio y fecha_cierre inclusive
- El vendedor de una subasta no puede pujar en su propia subasta
- El mismo usuario no puede realizar dos ofertas consecutivas en la misma subasta (debe esperar a que otro oferente puge primero)
- La primera oferta en una subasta debe alcanzar o superar el precio base
- Cada oferta posterior debe superar a la mayor oferta actual en al menos el incremento mínimo
- Cuando vence el plazo de cierre, la subasta queda "cerrada" y ya no admite más ofertas. Posteriormente se debe procesar el cierre para asignar el ganador (la mayor oferta), si lo hubiera

Toda esta información se almacena en las siguientes tres tablas:

USUARIO
email
ana.lopez@mail.com
bruno.martinez@mail.com
...

SUBASTA									
id	descripcion	categoria	email_vendedor	fecha_inicio	fecha_cierre	precio_base	incremento_min	email_ganador	monto_ganador
100	Cuadro al óleo San Telmo	Arte	elena.garcia@mail.com	2026-05-01 10:00:00	2026-07-15 18:00:00	50000.00	5000.00		
...	...	...	...	...	...	...	...	...	...

OFERTA				
id_subasta	nro_oferta	email_usuario	fecha_hora	monto
150	1	carla.perez@mail.com	2026-03-06 08:13:00	200000.00
...	...	...	...	...

Las restricciones de cada tabla son las siguientes:

- **USUARIO:** tiene un único campo, email. La tabla NO se carga desde un archivo: se auto completa mediante triggers cada vez que aparece un email_vendedor o email_usuario nuevo (ver puntos 4.b y 4.c)
- **SUBASTA:** Los campos email_ganador y monto_ganador admiten NULL hasta que se procesa el cierre de la subasta
- **OFERTA:** es una entidad débil cuya existencia depende de la SUBASTA a la que pertenece

La información es provista en 2 archivos CSV (Comma Separated Values) que se detallan a continuación.

a) Archivo subasta.csv

Contiene la información de las subastas publicadas. Las columnas del archivo son:

- **id:** identificador único de la subasta
- **descripcion:** descripción del ítem en subasta
- **categoria:** categoría del ítem (por ejemplo: Arte, Electrónica, Libros, Vehículos, Coleccionables)
- **email_vendedor:** email del usuario que publica la subasta
- **fecha_inicio:** fecha y hora de apertura de la subasta
- **fecha_cierre:** fecha y hora de cierre de la subasta
- **precio_base:** monto mínimo que debe alcanzar la primera oferta
- **incremento_min:** monto mínimo en que debe superarse la oferta actual para realizar una nueva oferta

Notar que en el archivo no se incluye información del ganador ni del monto ganador. Esos campos quedan en NULL al cargar el archivo y se completan posteriormente al procesar el cierre de cada subasta vencida.

b) Archivo oferta.csv

Contiene el historial de ofertas realizadas. Las columnas del archivo son:

- **id_subasta:** identificador de la subasta sobre la que se ofertó
- **email_usuario:** email del usuario que realizó la oferta
- **fecha_hora:** instante en que se realizó la oferta
- **monto:** monto ofertado

Notar que el archivo no incluye la columna nro_oferta: ese valor se asigna automáticamente a través de un trigger.

Adicionalmente, el administrador de la base de datos desea poder procesar el cierre de las subastas vencidas asignando el ganador y el monto, y generar reportes periódicos de actividad por categoría.

En resumen, la finalidad de este Trabajo Práctico Especial consiste en implementar lo antes descripto. Específicamente se debe hacer lo siguiente:

- Crear las 3 tablas USUARIO, SUBASTA y OFERTA
- Implementar un trigger sobre la tabla SUBASTA que autopueble la tabla USUARIO con el email_vendedor si éste todavía no existe
- Implementar un trigger sobre la tabla OFERTA que (a) autopueble la tabla USUARIO con el email_usuario si éste todavía no existe, (b) valide las reglas de negocio al insertar una nueva oferta, y (c) asigne automáticamente el número correlativo de la oferta
- Importar los datos provistos desde los archivos CVS a las tablas creadas
- Implementar la función cerrar_subasta para procesar el cierre y determinar el ganador y su monto
- Implementar la función reporte_subastas para generar el reporte periódico de actividad

4. Explicación paso a paso

a) Creación de las tablas USUARIO, SUBASTA y OFERTA

Deben crearse las 3 tablas USUARIO, SUBASTA y OFERTA con los tipos de datos adecuados para almacenar los datos procedentes de los archivos CSV. Definir las claves y las restricciones según corresponda, incluyendo las claves foráneas entre tablas y las restricciones de integridad que se desprendan del dominio de los campos.

b) Implementación del trigger de auto completa los vendedores

Debe implementarse un trigger sobre la tabla SUBASTA, que verifique si el email_vendedor de la nueva subasta ya existe en la tabla USUARIO; si no existe, debe insertarlo. Si el usuario ya existía, no debe hacer nada.

c) Implementación del trigger de validación e inserción de ofertas

Debe implementarse un trigger sobre la tabla OFERTA, que realice las siguientes acciones, en este orden:

- Auto completar la tabla USUARIO con el email_usuario de la nueva oferta si éste todavía no existe (idéntico criterio que el trigger del punto b)
- Validar que la subasta exista y que la fecha_hora de la oferta esté dentro del rango [fecha_inicio, fecha_cierre] de la subasta. En caso contrario, rechazar la operación
- Validar que el oferente no sea el vendedor de la subasta. En caso contrario, rechazar
- Validar que el oferente no haya realizado la última oferta de la subasta (no se permiten dos ofertas consecutivas del mismo usuario). En caso contrario, rechazar
- Validar el monto: si es la primera oferta, debe alcanzar o superar el precio_base; si no, debe superar a la mayor oferta actual en al menos el incremento_min. En caso contrario, rechazar
- Asignar automáticamente el campo nro_oferta como el siguiente número correlativo dentro de la subasta (el usuario no provee este valor en el INSERT)

Por ejemplo, asumiendo que la tabla SUBASTA contiene la siguiente fila:

SUBASTA									
id	descripcion	categoría	email_vendedor	fecha_inicio	fecha_cierre	precio_base	incremento_min	email_ganador	monto_ganador
100	Cuadro al óleo San Telmo	Arte	elena.garcia@mail.com	2026-05-01 10:00:00	2026-07-15 18:00:00	50000.00	5000.00		

y que la tabla OFERTA está vacía para la subasta 100.

- 1) Si el usuario realiza la operación: `INSERT INTO oferta (id_subasta, email_usuario, fecha_hora, monto) VALUES (100, 'carla.perez@mail.com', '2026-06-10 14:00:00', 50000);`

se inserta la oferta exitosamente con nro_oferta = 1 asignado por el trigger, quedando la tabla OFERTA de la siguiente manera:

OFERTA				
id_subasta	nro_oferta	email_usuario	fecha_hora	monto
100	1	carla.perez@mail.com	2026-06-10 14:00:00	50000.00

- 2) Si luego el usuario realiza la operación: `INSERT INTO oferta (id_subasta, email_usuario, fecha_hora, monto) VALUES (100, 'hernan.diaz@mail.com', '2026-06-10 15:00:00', 55000);`

se inserta la oferta con nro_oferta = 2, quedando la tabla OFERTA de la siguiente manera:

OFERTA				
id_subasta	nro_oferta	email_usuario	fecha_hora	monto
100	1	carla.perez@mail.com	2026-06-10 14:00:00	50000.00
100	2	hernan.diaz@mail.com	2026-06-10 15:00:00	55000.00

Por el contrario, las siguientes operaciones deben ser rechazadas con un mensaje de error claro (los mensajes son de ejemplo), sin alterar las tablas:

- 3) Misma subasta, mismo usuario que la última oferta (auto-puja):

```
INSERT INTO oferta (id_subasta, email_usuario, fecha_hora, monto)
VALUES (100, 'hernan.diaz@mail.com', '2026-06-10 16:00:00', 60000);
-- hernan.diaz@mail.com ya hizo la última oferta
```
- 4) Usuario que es el vendedor de la subasta:

```
INSERT INTO oferta (id_subasta, email_usuario, fecha_hora, monto)
VALUES (100, 'elena.garcia@mail.com', '2026-06-10 17:00:00', 65000);
-- elena.garcia@mail.com es la vendedora de la subasta 100
```
- 5) Monto insuficiente (no supera al actual más el incremento mínimo):

```
INSERT INTO oferta (id_subasta, email_usuario, fecha_hora, monto)
VALUES (100, 'lucas.acosta@mail.com', '2026-06-10 18:00:00', 57000);
-- Requiere al menos 60000
```
- 6) Subasta vencida (la fecha_hora excede fecha_cierre):

```
INSERT INTO oferta (id_subasta, email_usuario, fecha_hora, monto)
VALUES (150, 'lucas.acosta@mail.com', '2026-06-15 12:00:00', 300000);
-- La subasta 150 cerró el 30/04/2026
```

También se considera el rechazo si la primera oferta no alcanza al precio_base. En todos los casos, el trigger debe lanzar una excepción con un mensaje que explique con claridad la razón del rechazo.

d) Importación de los datos

Utilizando el comando COPY de PostgreSQL, se deben importar TODOS los datos de los archivos CSV en las tablas creadas en el punto a). Los archivos CSV provistos por la cátedra NO pueden ser modificados.

Notar que al cargar **subasta.csv** y **oferta.csv**, los triggers de los puntos b) y c) se ejecutarán auto completando la tabla USUARIO con cada email nuevo, validando reglas y asignando nro_oferta. Los datos provistos están contruidos de forma tal que ninguna fila será rechazada durante la importación.

e) Función cerrar_subasta

Debe implementarse una función PSM **cerrar_subasta(p_id)** que procese el cierre de una subasta cuyo plazo ya venció, asignando el ganador correspondiente. La función debe:

- Validar que la subasta exista; si no, rechazar con un mensaje claro
- Validar que el plazo de la subasta ya haya vencido (fecha_cierre menor o igual a la fecha actual del sistema). Si todavía está activa, rechazar con un mensaje claro
- Validar que la subasta aún no tenga ganador asignado. Si ya fue cerrada previamente con un ganador, rechazar con un mensaje claro
- Si todas las validaciones pasan: identificar la mayor oferta de la subasta. Actualizar los campos id_ganador y monto_ganador con esos valores en la tabla SUBASTA

- Si la subasta cerró sin haber recibido ninguna oferta, la función debe completarse normalmente sin modificar la subasta (y sin error). En este caso, una nueva invocación a cerrar_subasta sobre la misma subasta no debe fallar

Por ejemplo, asumiendo el estado de las tablas que resulta de cargar los CSVs provistos:

La invocación: `SELECT cerrar_subasta(150);`

procesa el cierre de la subasta 150 (vencida el 30/04/2026, con 6 ofertas registradas) y asigna como ganador al usuario con la mayor oferta, actualizando los campos correspondientes en la tabla SUBASTA:

SUBASTA (fragment de campos)				
id	Descripcion	fecha_cierre	email_ganador	monto_ganador
150	Notebook gamer i7 16GB	2026-04-30 23:00:00	victoria.reyes@mail.com	285000.00

Las siguientes invocaciones deben ser rechazadas con mensajes claros (los mensajes son de ejemplo):

`SELECT cerrar_subasta(100);` -- la subasta 100 todavía está abierta

`SELECT cerrar_subasta(150);` -- la subasta 150 ya fue cerrada con ganador

Y la siguiente invocación, sobre una subasta vencida sin ofertas (ej. la subasta 200), no debe asignar ganador ni dar error, incluso si se invoca varias veces:

`SELECT cerrar_subasta(200);`

`SELECT cerrar_subasta(200);`

f) Función `reporte_subastas`

Debe implementarse una función PSM `reporte_subastas(p_desde, p_categoria)` que genere un reporte de las subastas cuya fecha_cierre sea mayor o igual al parámetro p_desde (tipo DATE), opcionalmente filtrado por categoría (parámetro p_categoria, por defecto NULL para indicar todas las categorías).

La función debe usar un cursor explícito para recorrer las subastas y producir la salida con RAISE NOTICE, agrupando por categoría y mostrando:

- Un encabezado del reporte con la categoría (si fue filtrada) y la fecha desde
- Un encabezado por cada categoría que aparezca (en orden alfabético)
- Un renglón por cada subasta (de menor a mayor por id), indicando id, descripción, precio base, ganador y monto ganador (si existe) o cantidad de ofertas (si no tiene ganador asignado aún)
- Un subtotal por categoría con: cantidad de subastas, cuántas tienen ganador, y monto total recaudado en esa categoría
- Un total general al final con los mismos datos sumados

En caso de que no existieran subastas que cumplan los filtros, la función no debe mostrar nada (ni siquiera el encabezado).

Por ejemplo, considerando los datos provistos y luego de haber procesado el cierre de las subastas vencidas, la invocación: `SELECT reporte_subastas('2026-01-01':DATE, null);` produce un reporte con todas las subastas agrupadas por categoría, con subtotales y total general:

Nota: Cabe aclarar que `::` es el operador de casteo de PostgreSQL, que es equivalente a la función `CAST()` ya vista en clase.

===== REPORTE DE SUBASTAS =====

Desde: 2026-01-01

== Categoría: Arte ==

[#100] Cuadro al óleo San Telmo - base \$ 50000.00 -> sin ganador asignado (0 ofertas)

[#101] Acuarela paisaje patagónico - base \$ 280000.00 -> sin ganador asignado (4 ofertas)

[#102] Litografía firmada Quinquela - base \$ 588000.00 -> sin ganador asignado (7 ofertas)

[#103] Boceto carbonilla retrato - base \$ 500000.00 -> sin ganador asignado (0 ofertas)

[#104] Pintura abstracta acrílico - base \$ 226000.00 -> ganador ximena.aguirre@mail.com por \$ 237000.00 (7 ofertas)

[#105] Grabado vintage 1940 - base \$ 205000.00 -> sin ganador asignado (7 ofertas)

[#200] Escultura bronce mediana - base \$ 180000.00 -> sin ganador asignado (0 ofertas)

-- subtotal Arte: 7 subastas, 1 con ganador, \$ 237000.00 recaudado

== Categoría: Coleccionables ==

[...] (resto de subastas)

-- subtotal Coleccionables: 6 subastas, 1 con ganador, \$ 156000.00 recaudado

== Categoría: Electrónica ==

[#106] Cámara Canon EOS 80D - base \$ 541000.00 -> sin ganador asignado (7 ofertas)

[#107] Equipo audio vintage Marantz - base \$ 193000.00 -> sin ganador asignado (0 ofertas)

[#108] Tablet Samsung 11 pulgadas - base \$ 596000.00 -> ganador diego.sanchez@mail.com por \$ 610000.00 (7 ofertas)

[#109] Reloj smartwatch deportivo - base \$ 233000.00 -> ganador diego.sanchez@mail.com por \$ 241000.00 (3 ofertas)

[#110] Consola retro PlayStation 1 - base \$ 276000.00 -> sin ganador asignado (4 ofertas)

[#111] Auriculares Sennheiser HD - base \$ 235000.00 -> sin ganador asignado (8 ofertas)

[#150] Notebook gamer i7 16GB - base \$ 200000.00 -> ganador victoria.reyes@mail.com por \$ 285000.00 (6 ofertas)

-- subtotal Electrónica: 7 subastas, 3 con ganador, \$ 1136000.00 recaudado

== Categoría: Libros ==

[...] (resto de subastas)

-- subtotal Libros: 6 subastas, 1 con ganador, \$ 291000.00 recaudado

```
== Categoría: Vehículos ==
[...] (resto de subastas)
-- subtotal Vehículos: 5 subastas, 2 con ganador, $ 343000.00 recaudado
```

```
===== TOTAL: 31 subastas, 8 con ganador, $ 2163000.00 recaudado =====
```

La invocación con filtro de categoría: `SELECT reporte_subastas('2026-01-01'::DATE, 'Electrónica');` produce el mismo reporte restringido a la categoría 'Electrónica' (el encabezado incluye la categoría filtrada y la fecha desde, y el total general considera solo esa categoría).

```
===== REPORTE DE SUBASTAS =====
Categoría: Electrónica
Desde: 2026-01-01
```

```
== Categoría: Electrónica ==
[#106] Cámara Canon EOS 80D - base $ 541000.00 -> sin ganador asignado (7 ofertas)
[#107] Equipo audio vintage Marantz - base $ 193000.00 -> sin ganador asignado (0 ofertas)
[#108] Tablet Samsung 11 pulgadas - base $ 596000.00 -> ganador diego.sanchez@mail.com por
$ 610000.00 (7 ofertas)
[#109] Reloj smartwatch deportivo - base $ 233000.00 -> ganador diego.sanchez@mail.com por
$ 241000.00 (3 ofertas)
[#110] Consola retro PlayStation 1 - base $ 276000.00 -> sin ganador asignado (4 ofertas)
[#111] Auriculares Sennheiser HD - base $ 235000.00 -> sin ganador asignado (8 ofertas)
[#150] Notebook gamer i7 16GB - base $ 200000.00 -> ganador victoria.reyes@mail.com por $
285000.00 (6 ofertas)
-- subtotal Electrónica: 7 subastas, 3 con ganador, $ 1136000.00 recaudado
```

```
===== TOTAL: 7 subastas, 3 con ganador, $ 1136000.00 recaudado =====
```

La invocación con una fecha futura para la que no haya subastas:
`SELECT reporte_subastas('2030-01-01'::DATE, null);`
no produce error ni salida alguna (ni encabezado).

5. Entregables

Los alumnos deberán entregar los siguientes documentos:

- El script SQL llamado **funciones.sql** con el código necesario para crear las tablas, los triggers y las funciones del trabajo práctico
- Un informe de como máximo 3 páginas, que debe contener:
 - El rol de cada uno de los participantes del grupo. Si bien en el TP deben estar involucrados todos los integrantes, se debe asignar un rol de supervisión para cada una de las tareas. Mínimamente los roles son: encargado del informe, encargado de las funciones, encargado del trigger, encargado del funcionamiento global del TP y encargado de investigación
 - Todo lo investigado para realizar el TP
 - Las dificultades encontradas y cómo se resolvieron
 - También se debe detallar aquí el proceso de importación de los datos realizado

6. Evaluación

La evaluación del trabajo se llevará a cabo utilizando los parámetros establecidos en la rúbrica asociada a la actividad en el Campus.

Se tendrá en cuenta que las consultas, más allá del funcionamiento (lo cual es fundamental), sean genéricas. Los docentes ejecutarán el proceso usando el conjunto de datos entregado, pero podrán también hacer pruebas con otros conjuntos de datos de similares características para evaluar el funcionamiento en distintos escenarios.

El informe deberá estar completo y sin faltas de ortografía.

En caso de que el trabajo no cumpliera los requisitos básicos para ser aprobado, los alumnos serán citados en la fecha de recuperatorio para corregir los errores detectados, defender la corrección y/o para resolver algunos ejercicios adicionales.